

INSTALLING NOMAD-SERVER

Installation, first node and updates

Windows - Linux - macOS - Raspberry Pi

Version 0.1.3. Windows (also exe, no Python),
the installer for Linux and Pi, venv and Docker,
the first node, the web UI, updates and errors.

© Guide by F. - t.me/reticulumitalia ©

November 2026

BEFORE YOU START

This guide is for nomad-server version 0.1.3.

nomad-server runs many NomadNet nodes in one program, with a web page (the web UI) to watch and manage them: pages, files, counters, visitors, backups. Each node has its own folder, identity (its address on the network), pages and files.

You need:

- Python 3.8 or newer, with pip;
- Reticulum (RNS): the only package nomad-server needs, the installer puts it in by itself;
- git (recommended, not required): updating is then one command.

If you have never installed Python, get it from python.org first (on Windows tick "Add python.exe to PATH" in the installer). This guide assumes that "python3 --version" (on Windows "python --version") answers.

If rnsd is already running on the computer, the nodes use it. Otherwise nomad-server runs Reticulum by itself, with the interfaces of your ~/.reticulum/config.

0. WHICH WAY TO CHOOSE

There are several ways to install nomad-server. Pick ONE:

a) WINDOWS: rns with pip and the start.bat file. Section 2.

b) LINUX AND RASPBERRY PI, INSTALLER (recommended for a server): a script that sets everything up, also the service that starts it at boot. Section 3.

c) IN A VENV: like the others, but rns stays in a folder of its own, without touching the system Python. Section 4.

d) DOCKER: nomad-server in a container. Section 5.

e) WINDOWS WITHOUT PYTHON: the ready-made programs (exe). Section 2.5.

In every case the first step is downloading nomad-server (section 1). After the install: the first node is in section 6.

1. DOWNLOADING NOMAD-SERVER

1.1 With git (recommended)

Open the terminal in the folder where you want it and:

```
git clone https://github.com/fr33n0w/nomad-server
cd nomad-server
```

If git is missing:

```
Windows:  https://git-scm.com/download/win  (install it, then
          open a NEW terminal)
Linux:    sudo apt install git
macOS:    git --version  (if it is missing, the Mac offers to
          install the "Command Line Tools": accept)
```

1.2 Without git (zip)

- Go to https://github.com/fr33n0w/nomad-server
- Green "Code" button > "Download ZIP".
- Extract the zip into a folder of your choice.
- Open the terminal INSIDE that folder (the one with nomad_server.py in it).

Your nodes, settings and counters live in this folder: when you update from a zip, keep server.cfg, nodes/ and data/ (section 8).

2. WINDOWS

2.1 Installing Reticulum

In the Command Prompt, in the nomad-server folder:

```
python -m pip install rns
```

(if "python" is missing: py -m pip install rns)

2.2 Starting it

Double-click start.bat: nomad-server starts and the web UI opens in a window of its own (Edge or Chrome). Closing that window stops the server.

Other ways, from the Command Prompt in the folder:

```
python nomad_server.py          log in the terminal, Ctrl+C stops it
python nomad_server.py --app    the web UI in its own window
```

start_hidden.vbs starts it with no window at all: open the web UI at http://127.0.0.1:8484 and stop it with the "Stop server" button in Settings, or with:

```
python nomad_server.py --stop
```

2.3 Desktop and Start menu icons

Settings > Shortcuts puts a nomad-server icon on the desktop and in the Start menu. The icon starts THIS installation (this folder) in its window; if it is already running it only opens the window.

There is one nomad-server icon per computer: if you have more than one installation (a test copy, say), the icon starts the last one that pressed Shortcuts. To move it to another one, press Shortcuts again from that one.

2.4 Common errors on Windows

- **"python" is not recognised**

  Python is missing or not in the PATH: reinstall it from python.org and tick "Add python.exe to PATH", or use "py" instead of "python".

- **"No module named RNS"**

  rns is installed in another Python (or in a venv you did not use to start it). Install it with the same Python you start nomad-server with:

```
python -m pip install rns
```

- **Windows asks for Firewall permission at the first start**

  Allow it for private networks if you want to open the web UI from your phone or another PC at home (section 6.4).

2.5 Ready-made programs (exe): no Python needed

On the Releases page (github.com/fr33n0w/nomad-server/releases) there are two zips for Windows. Extract one and double-click the exe inside; nothing else to install, not even Python:

```
NomadServer-CLI-v0.1.3-win.exe
    the server with its terminal (log on screen), like
    python nomad_server.py
NomadServer-APP-v0.1.3-win.exe
    no terminal: the web UI in a window of its own;
    closing it stops the server
```

WHERE YOUR DATA GOES: each program keeps server.cfg, nodes\ and data\ in its OWN folder, the one with the exe. A copy extracted somewhere else starts as a new, empty server, with new nodes and new addresses: ten folders, ten separate servers.

To always use the same server, give its folder with --base:

```
NomadServer-APP-v0.1.3-win.exe --base D:\nomad-data
NomadServer-CLI-v0.1.3-win.exe --base D:\nomad-data
```

In a desktop shortcut: right click > Properties, and add --base D:\nomad-data at the end of "Target". --base also takes the folder of an existing installation (the one with nomad_server.py), with its nodes and settings: stop that server first.

Only one server runs on a folder: starting the app again while it runs only opens another window. The icons made by Settings > Shortcuts already carry the folder of the server that made them.

UPDATING: the version is in the file name, so the zip of a new version extracted somewhere else is a new, empty server. Extract it over the old folder (server.cfg, nodes\ and data\ stay), or keep your data in a folder of its own with --base. Then press Settings > Shortcuts again: the icons still point to the old exe.

3. LINUX AND RASPBERRY PI: THE INSTALLER (install.sh)

3.1 Running it

On Debian, Ubuntu and Raspberry Pi OS:

```
sudo apt install git python3-pip
git clone https://github.com/fr33n0w/nomad-server
cd nomad-server
bash install.sh
```

Run it as your normal user, not as root: it uses sudo only to install the service.

3.2 What it asks

1) If rns is missing and no venv is in use: whether to install it for your user (pip --user). Answer n to install nothing and use a venv instead (section 4).
2) If the web UI port (8484) is taken: another port.
3) Whether to open the web UI to the local network (0.0.0.0); if yes, a password (strongly suggested; empty = none).
4) The name of your first node (only if there are none yet).
5) Whether to install the systemd service "nomad-server": it starts at boot, as your user, from this folder, after rnsd when rnsd.service exists. If "nomad-server" is already used by another folder, it asks a name for this one (for example nomad-server-2): several servers can run side by side.

Running it again is safe: it keeps nodes, settings and service name, and restarts the service.

3.3 Living with the service

State of the server (nodes, interfaces, visitors connected now):

```
python3 nomad_server.py --status
```

The log, live:

```
journalctl -u nomad-server -f
```

Stop, start, restart:

```
sudo systemctl stop nomad-server
sudo systemctl start nomad-server
sudo systemctl restart nomad-server
```

Under systemd "--stop" is refused (systemd would start it again): use systemctl. Restart from the web UI works.

Removing the service:

```
sudo systemctl disable --now nomad-server
sudo rm /etc/systemd/system/nomad-server.service
sudo systemctl daemon-reload
```

3.4 Common errors on Linux

- **"python3 not found"**

  Install it:

  ```
  sudo apt install python3 python3-pip
  ```

- **"externally-managed-environment" with pip install rns by hand**

  Debian 12+, Ubuntu 23.04+ and Pi OS protect the system Python. Use the installer (it installs for your user), or a venv (section 4).

- **The status in the web UI is empty, no visitor names (the log says "rnsd" refuses the requests)**

  rnsd runs under another user: see section 9.

4. IN A VENV

A venv keeps rns and its libraries inside the nomad-server folder, apart from the system Python. On Debian, Ubuntu and Pi OS first:

```
sudo apt install git python3-venv
```

4.1 With the installer (Linux, Pi): starts at boot

In the nomad-server folder:

```
python3 -m venv venv
bash install.sh
```

The installer finds venv (or .venv) in the folder by itself, also when it is not activated: it installs rns in it and the service runs with the venv python. Then everything is as in section 3, with the venv python for the commands:

```
venv/bin/python nomad_server.py --status
```

4.2 By hand (Linux, macOS, Pi)

```
python3 -m venv venv
venv/bin/pip install rns
venv/bin/python nomad_server.py
```

4.3 By hand (Windows)

```
python -m venv venv
venv\Scripts\pip install rns
venv\Scripts\python nomad_server.py
```

Use the venv python for every command (--status, --stop, --app, ...), or activate the venv once in the terminal (Linux: source venv/bin/activate, Windows: venv\Scripts\activate) and then plain "python". Icons made with Settings > Shortcuts use the venv by themselves.

4.4 Where Reticulum runs

The venv always needs its own rns: it is the library nomad-server is made of. The network itself:

- rnsd already running for your user: nothing to do, nomad-server connects to it, and reconnects when rnsd restarts. Keep the two rns close in version (pip install -U rns on both).
- no rnsd: nomad-server runs Reticulum by itself with the interfaces of ~/.reticulum/config. Other programs (NomadNet, rBrowser) can use it while it runs.

5. DOCKER

5.1 Starting it

In the nomad-server folder:

```
mkdir nomad-data
NOMAD_PASSWORD=yourpassword docker compose up -d
```

The web UI is on port 8484. Everything else stays in nomad-data/: settings, nodes, identities, counters, logs and the Reticulum config.

5.2 The Reticulum config

At the first start it is created in nomad-data/reticulum/config: add an interface there (a TCP server such as rmap.world, for example), because inside Docker AutoInterface finds no peers. The comments in docker-compose.yml show how to use the rnsd of the host instead.

5.3 Commands

```
docker compose exec nomad-server python /app/nomad_server.py --base /data --status
docker compose stop
```

"Stop server" in the web UI is started again by Docker: use docker compose stop.

6. FIRST START: YOUR FIRST NODE

6.1 Opening the web UI

In your browser: http://127.0.0.1:8484

(start.bat and --app open it by themselves.)

6.2 Creating a node

- Press "Create new node" and give it a name: it gets a new identity, that is a new address on the network, and a sample index.mu page.
- New nodes start STOPPED: nothing goes online before it is ready.
- Put your pages in nodes/<node>/pages/ (index.mu is the home page) and the downloads in nodes/<node>/files/. Or use "Edit pages" on the node: a file manager with an editor, a saved page is served at once.
- Press "Start": the node goes online and announces itself.

The node's "identity" file is its private key: keep it private, and save it with "Backup node" (Settings).

6.3 Is it reachable?

The Overview shows the Reticulum interfaces: at least one must be up, otherwise nobody reaches the node. Check with:

```
rnstatus
```

A node is found by the others only after it announces: after a restart it announces again by itself (Announce at start).

Below: "Live Clients Connections" = who is connected now (a closed link stays greyed for a minute) and "Incoming Connections History" = every finished visit, newest first, with the pages seen ("Load more" for older ones, "Clear history" to empty it).

6.4 The web UI from your phone or another PC

By default the web UI answers only on this computer (127.0.0.1). To open it to your home network: Settings > Server, "Web UI address" = 0.0.0.0, and set a password in the same page ("Web UI password"): whoever reaches the web UI can manage the nodes. Then open http://COMPUTER-ADDRESS:8484 from the phone.

The password can also be set from the terminal:

```
python3 nomad_server.py --set-password
```

With the server running it applies at once, no restart (handy over ssh).

Set it on THIS computer: from another device the first password is refused (a stranger could set one and lock you out). Over the network, without a password, editing pages and files, importing nodes, identities and backups are refused: they touch files on this computer.

Alone at home? Settings > Server > "Trust local network": the first password can also be set from a device on your home network (192.168.x, 10.x, 172.16-31.x). Turn it on only if you trust everyone on the network and Wi-Fi; it can be changed only from this computer or with the password.

6.5 A node you already have

A node's address comes from its "identity" file (the private key). Give it to a node of nomad-server and the node keeps its address: whoever knew it finds it again.

THE QUICK WAY: Overview > "Import existing node".

- "Autoscan import" searches the computer running nomad-server for old nodes (NomadNet, rns_server_page.py and similar: an identity file and pages/ with .mu pages) and lists them with name, path, pages and address. Pick one.
- "Manual import": open the old folder yourself in the folder explorer of the window (for NomadNet: .nomadnetwork in your home folder). "Browse folders on this computer" opens the system's folder window; from another PC or the phone it is "Browse folders on your device", and the folder you choose is sent to the server (only identity, settings, pages and files).
- The name comes from the old config and can be changed. Then "Import node": identity, pages, files and old counters are copied into a new node, which stays stopped. The old folder is not touched. Without .mu pages there is nothing to import.
- Then do steps 5 and 6 below: stop the old node, Start the new one.

BY HAND. What you need from the old node (NomadNet):

```
identity   ~/.nomadnetwork/storage/identity
pages      ~/.nomadnetwork/storage/pages/
files      ~/.nomadnetwork/storage/files/
```

Sometimes NomadNet uses ~/.config/nomadnetwork/ instead of ~/.nomadnetwork/. On Windows ~ is C:\Users\<you>. If the NomadNet config has pages_path or files_path in the [node] section, the pages and files are there.

A node made with rns_server_page.py or similar: identity, pages/ and files/ are in its folder, and data/visits.json holds its counters.

The steps:

1. "Create new node", with the old node's name. It starts stopped, so its new identity never goes on the network.
2. Settings > Node > "Import identity", choose the old identity file. The address shown under the node becomes the old one: check it is the one you know.
3. Copy the old pages into nodes/<node>/pages/ and the files into nodes/<node>/files/. Settings shows where the folders are, "Open folder" opens them. The old index.mu replaces the sample one. On Linux copy with:

```
cp -a ~/.nomadnetwork/storage/pages/. nodes/<node>/pages/
```

   so executable pages stay executable.
4. Optional: copy data/visits.json into nodes/<node>/data/. The old counts are added to the new ones, once.

5. Stop the old node.
   - NomadNet: in its config, section [node], set

     enable_node = no

   and restart it. NomadNet stays your messenger with the same LXMF address: only the node moves.
   - Another server: stop it and disable its service (sudo systemctl disable --now NAME).
6. Press "Start" on the node, then "Announce". Open it from another computer (NomadNet, MeshChat, rBrowser) to check.

Never serve the same node from two places at once: the network would see two nodes with one address.

If a program regenerates the pages, make it write into nodes/<node>/pages/. If the old server had its own code for dynamic pages, it becomes the node's plugin.py (6.6).

6.6 Dynamic pages

Normal .mu pages are fixed text. For pages that change (guestbooks, forms, live data) there are two ways.

Executable pages, as in NomadNet: a .mu that starts with

```
#!/usr/bin/env python3
```

and, on Linux, made executable (chmod +x page.mu). At each request the program runs and what it prints is the page. Form fields arrive as environment variables (field_*, var_*, remote_identity). On Windows the first line is enough.

Plugin: a plugin.py file in the node folder (nodes/<node>/plugin.py). It is loaded inside nomad-server when the node starts, so no process is started for each request. A complete example:

```
PATHS = ["/page/hello.mu"]

def start(node):
    node.log("plugin ready")

def page(path, data, who=None, who_name="", link_id=None):
    return ">Hello\n\nHello " + (who_name or "stranger") + "\n"

def stop():
    pass
```

- PATHS: the pages the plugin makes. They win over a file with the same name.
- start(node): runs once when the node starts. Start threads or background programs here, like a page generator.
- page(...): makes the page. data holds the form fields; who is the visitor's LXMF address if they identified, who_name their name. Returns micron text.
- stop(): when the node stops, stop what start started.

After changing plugin.py press "Restart" on the node. Errors go to the node's log (Logs tab). The README explains the rest (paths(), handles(), node.cfg).

7. USEFUL COMMANDS

In the nomad-server folder (Windows: python instead of python3, venv: venv/bin/python):

```
python3 nomad_server.py
```

Starting it (all optional):

```
--app                the web UI in its own window
--service            quiet: only warnings and errors, the
                     full log in logs/server.log
--no-webui           no web interface
--base DIR           the folder with server.cfg, nodes/ and
                     data/ (default: next to the program)
```

Commands to the server already running:

```
--status             nodes, interfaces, visitors connected
--log                the last 60 lines of the log
--log -f             and the new ones as they come
--stop               stop it (not under systemd)
--block LXMF         block a peer on every node
--unblock LXMF       remove it from the blocklist

--set-password       the web UI password (empty = none),
                     applies at once
```

On the files (restart the server afterwards):

```
--new-node NAME      a new node with a new identity
```

The full list: python3 nomad_server.py --help

Only one server runs on a folder: the same identities can never be online twice. Started again with --app, it just opens the window.

8. UPDATING NOMAD-SERVER

8.1 Installed with git clone

```
git pull
```

then restart it: "Restart" in the web UI, or with the service:

```
sudo systemctl restart nomad-server
```

In a venv, update Reticulum too:

```
venv/bin/pip install -U rns
```

8.2 Installed from a zip or a tar

Download the new version and extract it OVER the old folder: server.cfg, nodes/ and data/ are not in the archive, so they stay as they are. On a Linux server, from the folder that holds nomad-server:

```
wget -O ns.tar.gz https://github.com/fr33n0w/nomad-server/archive/refs/heads/main.tar.gz
tar xzf ns.tar.gz --strip-components=1 -C nomad-server
rm ns.tar.gz
sudo systemctl restart nomad-server
```

8.3 Docker

```
git pull
docker compose up -d --build
```

9. COMMON ERRORS AT START

- **"nomad-server already runs on this folder"**

  A server is already running on this folder (maybe hidden, started by start_hidden.vbs or by the service). Use that one, or stop it first:

  ```
  python3 nomad_server.py --stop
  ```

- **The web UI port is already in use**

  Another program (or another nomad-server) uses port 8484: change "port" in the [webui] section of server.cfg, then start it again. The installer asks for another port itself.

- **The web UI does not open from the phone**

  - Is "Web UI address" 0.0.0.0 (section 6.4)?
  - Are the phone and the computer on the same Wi-Fi network?
  - Windows: the Firewall must allow Python on private networks.

- **Nobody reaches the node**

  - Is the node started (Start) and has it announced?
  - rnstatus: at least one interface must be up.

- **rnsd under another user (no Reticulum status in the web UI, no visitor names)**

  rnsd answers only programs with its key: pages are served anyway, but its status requests are refused. As the user that runs rnsd, this prints the key:

  ```
  python3 -c "import RNS, os
  p = os.path.expanduser('~/.reticulum/storage/transport_identity')
  k = RNS.Identity.from_file(p).get_private_key()
  print(RNS.Identity.full_hash(k).hex())"
  ```

  Then in ~/.reticulum/config of nomad-server's user, under [reticulum], add the line:

```
rpc_key = THE-KEY
```

and restart nomad-server. The key gives control of rnsd: keep it private.

Honeypot nodes (bait nodes against crawlers) are an advanced function: they are explained in the README on GitHub.

QUICK SUMMARY

```
1. Download:  git clone https://github.com/fr33n0w/nomad-server
              cd nomad-server
2. Install:   Windows: python -m pip install rns
              Linux, Pi: bash install.sh
              venv: python3 -m venv venv, then bash install.sh
              Docker: mkdir nomad-data, docker compose up -d
3. Start:     Windows: start.bat
              Linux: python3 nomad_server.py (or the service)
4. Open:      http://127.0.0.1:8484
5. Node:      Create new node, pages in nodes/<node>/pages/,
              then Start. A node you already have:
              Import existing node.
6. Network:   rnstatus: at least one interface up.
7. Update:    git pull, then Restart.
```

USEFUL LINKS

nomad-server on GitHub

https://github.com/fr33n0w/nomad-server

Python (download)

https://www.python.org/downloads/

Git for Windows

https://git-scm.com/download/win

NomadNet

https://github.com/markqvist/NomadNet

rmap.world (map of the Reticulum network)

https://rmap.world

Official Reticulum manual

https://reticulum.network/manual/

Reticulum Italia on Telegram

https://t.me/reticulumitalia

© Guide by F. - t.me/reticulumitalia ©